Issue #71 — content_mode replication through master, dedup, search, fixtures

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Replicate the `content_mode` fact (`full`/`search-only`/`metadata-only`, introduced by #70) from each per-source index into the master catalog, surface it on `master list`, `dedup --json`, and `search --all --json`, and close the dedup silent-empty gap where metadata-only archives vanish from a report with no explanation.

Architecture: `content_mode` already lives in each per-source index's `meta` table (#70). This plan replicates that one fact into the master's `archives` table at `Master::add()` time (same probe-then-ALTER migration shape as `path_raw`, #37/ADR 0002), threads it onto `ArchiveRow`/`ReportArchive`/`FederatedHit`, and uses it in two new places: `dedup`'s `summary.skipped_archives` (metadata-only archives currently vanish from `fetch_scope`'s `WHERE content_hash IS NOT NULL` filter with zero explanation) and `search --all`'s per-archive JSON (`mode` field, `search --json` already has one from #70).

Tech Stack: Rust, rusqlite, serde_json, clap — no new dependencies.

Spec: GitHub issue #71 (`gh issue view 71`), child of #39. Depends on #70 (merged, PR #72) which defined `ContentMode` in `src/indexer.rs` and the read-time gates in `src/searcher.rs`/`src/main.rs`.

Global Constraints

- The dedup/search/master JSON surfaces are a frozen public contract (`docs/CONTRACT.md`): every change in this plan is **additive-only** — new fields, never renamed or removed ones. Verified by `tests/contract.rs` golden fixtures under `BACKUPSAGE_BLESS=1`.
- BackupSage never rewrites or deletes from an archive (safety invariant, restated in the issue body — nothing in this plan touches archive bytes, only the master catalog and reports).
- New master catalogs get the `content_mode` column in `CREATE TABLE` directly; catalogs opened from an on-disk file predating this change get it via probe-then-`ALTER TABLE` at `open_at()`, exactly like `path_raw` (`src/master.rs:193-195`).
- Absent source meta (pre-#39/#70 indexes, and v2 indexes which predate the whole concept) replicates as `"full"` — the same grandfather rule `store::content_mode` already applies when reading a per-source index directly.
- `cargo clippy --all-targets -- -D warnings` and `cargo fmt --check` must stay clean after every task (repo gate, see `./dev gate` if present, else run both directly).

Task 1: Replicate `content_mode` into the master catalog schema

Files: - Modify: `src/master.rs:29-43` (`ArchiveRow`), `src/master.rs:228-287` (`init_master_conn`), `src/master.rs:167-207` (`open_at` migration), `src/master.rs:297-346` (`SourceIdentity` / `read_identity`), `src/master.rs:375-484` (`Master::add`), `src/master.rs:540-565` (`Master::list`) - Test: `tests/master.rs` (new tests appended)

Interfaces: - Produces: `ArchiveRow.content_mode: String` (one of `"full"`, `"search-only"`, `"metadata-only"`) — consumed by Task 2 (rendering) and Task 3 (dedup skip logic). - Consumes: `crate::store::content_mode(&Connection) -> ContentMode` (existing, `src/store.rs:97-100`), `crate::indexer::ContentMode::as_str()` (existing, `src/indexer.rs:70-74`).

- [ ] **Step 1: Write the failing test — content_mode replicates and defaults to full**

Append to `tests/master.rs`:

```
#[test]
fn add_replicates_content_mode() {
    let dir = tempfile::tempdir().unwrap();
    let db = index_archive(dir.path(), "plain.tar", &[("a.txt", b"hi".to_vec())]);
    let mut m = open_master(dir.path());
    m.add(&db).unwrap();
    assert_eq!(m.list().unwrap()[0].content_mode, "full");
}

#[test]
fn search_only_and_metadata_only_content_mode_replicate() {
    let dir = tempfile::tempdir().unwrap();
    for (name, mode, expect) in [
        ("so.tar", indexer::ContentMode::SearchOnly, "search-only"),
        ("mo.tar", indexer::ContentMode::MetadataOnly, "metadata-only"),
    ] {
        let archive = write_archive(dir.path(), name, &build_tar(&[("f.txt",
b"x".to_vec())]));
        let opts = IndexOptions {
            mode,
            ..IndexOptions::default()
        };
        let db = indexer::run_index(&archive, None, &opts).unwrap().db_path;
        let mut m = open_master(dir.path());
        m.add(&db).unwrap();
        let row = m
            .list()
            .unwrap()
            .into_iter()
            .find(|r| r.label == name)
            .unwrap();
        assert_eq!(row.content_mode, expect);
    }
}

/// Migration precedent: a pre-#39 v3 index (no `content_mode` meta key at
/// all — the state of every index built before #70) must grandfather to
/// `full` all the way through master replication, exactly as
/// `store::content_mode` already grandfathers it on direct reads
/// (`tests/modes.rs::old_index_without_mode_key_reads_as_full`).
#[test]
fn pre_70_index_without_content_mode_key_replicates_as_full() {
    let dir = tempfile::tempdir().unwrap();
    let db = index_archive(dir.path(), "old.tar", &[("a.txt",
b"grandfather".to_vec())]);
    {
        let conn = rusqlite::Connection::open(&db).unwrap();
        conn.execute("DELETE FROM meta WHERE key='content_mode'", [])
            .unwrap();
    }
    let mut m = open_master(dir.path());
    m.add(&db).unwrap();
```

```
    assert_eq!(m.list().unwrap()[0].content_mode, "full");
}

/// A master catalog file created before this change (no `content_mode`
/// column at all) must migrate on open, same probe-then-ALTER shape as
/// `path_raw` (#37/ADR 0002) — never refuse to open an old master.
#[test]
fn legacy_master_without_content_mode_column_migrates_on_open() {
    let dir = tempfile::tempdir().unwrap();
    let master_path = dir.path().join("master.db");
    {
        // Fabricate a pre-#71 master: signed, but archives lacks the column.
        let conn = rusqlite::Connection::open(&master_path).unwrap();
        conn.pragma_update(None, "application_id", master::MASTER_APPLICATION_ID)
            .unwrap();
        conn.execute_batch(
            "CREATE TABLE archives (
                archive_id     INTEGER PRIMARY KEY,
                index_uuid     TEXT UNIQUE NOT NULL,
                db_path        TEXT NOT NULL,
                source_path    TEXT NOT NULL,
                source_type    TEXT NOT NULL,
                label          TEXT NOT NULL,
                schema_version INTEGER NOT NULL,
                files_count    INTEGER NOT NULL DEFAULT 0,
                completed      INTEGER NOT NULL DEFAULT 0,
                indexed_unix   INTEGER,
                archive_size   INTEGER,
                archive_mtime_unix INTEGER,
                archive_blake3 TEXT,
                db_size        INTEGER,
                db_mtime_unix  INTEGER,
                phash_algo     TEXT,
                status         TEXT NOT NULL DEFAULT 'ok',
                added_unix     INTEGER NOT NULL,
                synced_unix    INTEGER
            );
            CREATE TABLE files (
                archive_id   INTEGER NOT NULL,
                file_id      INTEGER NOT NULL,
                path         TEXT NOT NULL,
                entry_type   TEXT NOT NULL,
                kind         TEXT NOT NULL,
                size         INTEGER,
                mtime_unix   INTEGER,
                exif_unix    INTEGER,
                exif_src     TEXT,
                content_hash BLOB,
                phash        INTEGER,
                img_w        INTEGER,
                img_h        INTEGER,
                flags        INTEGER NOT NULL DEFAULT 0,
                PRIMARY KEY (archive_id, file_id)
```

```
            );",
        )
        .unwrap();
    }
    let db = index_archive(dir.path(), "new.tar", &[("a.txt", b"data".to_vec())]);
    let mut m = master::open_at(&master_path).unwrap();
    m.add(&db).unwrap();
    assert_eq!(m.list().unwrap()[0].content_mode, "full");
}
```

- [ ] **Step 2: Run tests to verify they fail**

Run: `cargo test --test master content_mode -- --test-threads=1` Expected: compile error — `ArchiveRow` has no field `content_mode` yet.

- [ ] **Step 3: Add the column to the schema (`init_master_conn`)**

In `src/master.rs`, inside `init_master_conn`'s `CREATE TABLE archives (...)` (around line 251), add the new column right after `phash_algo`:

```
            phash_algo     TEXT,
            content_mode   TEXT NOT NULL DEFAULT 'full',
            status         TEXT NOT NULL DEFAULT 'ok',
```

- [ ] **Step 4: Migrate existing on-disk masters (`open_at`)**

In `src/master.rs`, in `open_at`, right after the existing `path_raw` migration block (around line 193-195):

```
            if !crate::searcher::has_column(&conn, "files", "path_raw") {
                conn.execute_batch("ALTER TABLE files ADD COLUMN path_raw BLOB")?;
            }
            if !crate::searcher::has_column(&conn, "archives", "content_mode") {
                conn.execute_batch(
                    "ALTER TABLE archives ADD COLUMN content_mode TEXT NOT NULL
DEFAULT 'full'",
                )?;
            }
```

- [ ] **Step 5: Read `content_mode` from the source index (`SourceIdentity` / `read_identity`)**

In `src/master.rs`, add a field to `SourceIdentity` (around line 297-308):

```
struct SourceIdentity {
    index_uuid: String,
    schema_version: i64,
    source_path: String,
    source_type: String,
    completed: bool,
    indexed_unix: Option<i64>,
    archive_size: Option<i64>,
    archive_mtime_unix: Option<i64>,
    archive_blake3: Option<String>,
    phash_algo: Option<String>,
    content_mode: crate::indexer::ContentMode,
}
```

And populate it in `read_identity` (around line 332-344), reusing the existing grandfather-to-`Full` logic in `store::content_mode` rather than re-deriving it:

```
    let id = SourceIdentity {
        index_uuid,
        schema_version: version,
        source_path,
        source_type: searcher::get_meta(&conn, "source_type").unwrap_or_else(||
"tar".into()),
        completed: searcher::get_meta(&conn, "completed").as_deref() == Some("1"),
        indexed_unix: created.and_then(|v| v.parse().ok()),
        archive_size: searcher::get_meta(&conn, "archive_size").and_then(|v|
v.parse().ok()),
        archive_mtime_unix: searcher::get_meta(&conn, "archive_mtime_unix")
            .and_then(|v| v.parse().ok()),
        archive_blake3: searcher::get_meta(&conn, "archive_blake3"),
        phash_algo: searcher::get_meta(&conn, "phash_algo"),
        content_mode: store::content_mode(&conn),
    };
```

- [ ] **Step 6: Write `content_mode` in `Master::add` (INSERT and UPDATE)**

In `src/master.rs`, `Master::add` (around line 410-469), extend both the `UPDATE` and `INSERT` statements. Replace the existing `match existing { ... }` block with:

```
        let archive_id = match existing {
            Some(aid) => {
                self.conn.execute(
                    "UPDATE archives SET index_uuid=?1, db_path=?2, source_path=?3,
                        source_type=?4, label=?5, schema_version=?6, completed=?7,
                        indexed_unix=?8, archive_size=?9, archive_mtime_unix=?10,
                        archive_blake3=?11, db_size=?12, db_mtime_unix=?13,
                        phash_algo=?14, content_mode=?15, status=?16, synced_unix=?
17
                     WHERE archive_id=?18",
                    params![
                        id.index_uuid,
                        db_abs,
                        id.source_path,
                        id.source_type,
                        label,
                        id.schema_version,
                        id.completed as i64,
                        id.indexed_unix,
                        id.archive_size,
                        id.archive_mtime_unix,
                        id.archive_blake3,
                        db_size,
                        db_mtime,
                        id.phash_algo,
                        id.content_mode.as_str(),
                        status,
                        now_unix(),
                        aid
                    ],
                )?;
                aid
            }
            None => {
                self.conn.execute(
                    "INSERT INTO archives (index_uuid, db_path, source_path,
source_type,
                        label, schema_version, completed, indexed_unix,
archive_size,
                        archive_mtime_unix, archive_blake3, db_size, db_mtime_unix,
                        phash_algo, content_mode, status, added_unix, synced_unix)
                     VALUES (?1,?2,?3,?4,?5,?6,?7,?8,?9,?10,?11,?12,?13,?14,?15,?
16,?17,?17)",
                    params![
                        id.index_uuid,
                        db_abs,
                        id.source_path,
                        id.source_type,
                        label,
                        id.schema_version,
                        id.completed as i64,
                        id.indexed_unix,
```

```
                        id.archive_size,
                        id.archive_mtime_unix,
                        id.archive_blake3,
                        db_size,
                        db_mtime,
                        id.phash_algo,
                        id.content_mode.as_str(),
                        status,
                        now_unix()
                    ],
                )?;
                self.conn.last_insert_rowid()
            }
        };
```

- [ ] **Step 7: Expose it on `ArchiveRow` and `Master::list`**

In `src/master.rs`, add the field to `ArchiveRow` (around line 29-43):

```
#[derive(Debug, Clone)]
pub struct ArchiveRow {
    pub archive_id: i64,
    pub index_uuid: String,
    pub db_path: String,
    pub source_path: String,
    pub source_type: String,
    pub label: String,
    pub schema_version: i64,
    pub files_count: i64,
    pub completed: bool,
    pub indexed_unix: Option<i64>,
    pub archive_blake3: Option<String>,
    pub status: String,
    pub content_mode: String,
}
```

And update `Master::list` (around line 540-565):

```
    pub fn list(&self) -> Result<Vec<ArchiveRow>> {
        let mut stmt = self.conn.prepare(
            "SELECT archive_id, index_uuid, db_path, source_path, source_type,
label,
                    schema_version, files_count, completed, indexed_unix,
archive_blake3, status,
                    content_mode
             FROM archives ORDER BY archive_id",
        )?;
        let rows = stmt
            .query_map([], |r| {
                Ok(ArchiveRow {
                    archive_id: r.get(0)?,
                    index_uuid: r.get(1)?,
                    db_path: r.get(2)?,
                    source_path: r.get(3)?,
                    source_type: r.get(4)?,
                    label: r.get(5)?,
                    schema_version: r.get(6)?,
                    files_count: r.get(7)?,
                    completed: r.get::<_, i64>(8)? == 1,
                    indexed_unix: r.get(9)?,
                    archive_blake3: r.get(10)?,
                    status: r.get(11)?,
                    content_mode: r.get(12)?,
                })
            })?
            .collect::<std::result::Result<Vec<_>, _>>()?;
        Ok(rows)
    }
```

- [ ] **Step 8: Run tests to verify they pass**

Run: `cargo test --test master` Expected: PASS (all existing `tests/master.rs` tests plus the four new ones — the compiler will also point out any other `ArchiveRow { ... }` literal in the crate that now needs the new field; there are none outside `master.rs` itself).

- [ ] **Step 9: Commit**

```
git add src/master.rs tests/master.rs
git commit -m "feat(#71): replicate content_mode into the master catalog"
```

Task 2: Surface `content_mode` on `master list` (table + `--json`)

Files: - Modify: `src/main.rs:285-308` (`MasterCommands::List` JSON), `src/main.rs:699-730` (`print_master_table`) - Test: `tests/cli.rs` (new test appended)

Interfaces: - Consumes: `ArchiveRow.content_mode` (Task 1). - Produces: `master list --json` array entries gain `"content_mode"` key; `master list` table gains a `Mode` column. Both additive/cosmetic — no existing key renamed.

- [ ] **Step 1: Write the failing test**

Append to `tests/cli.rs`:

```
#[test]
fn master_list_json_carries_content_mode() {
    let dir = tempfile::tempdir().unwrap();
    let master = dir.path().join("m.db");
    let src = write_archive(
        dir.path(),
        "modeit.tar",
        &build_tar(&[("a.txt", b"data".to_vec())]),
    );
    run_ok(&[
        "index",
        src.to_str().unwrap(),
        "--mode",
        "search-only",
    ]);
    run_ok(&[
        "--master",
        master.to_str().unwrap(),
        "master",
        "add",
        dir.path().join("modeit.tar.db").to_str().unwrap(),
    ]);
    let out = run_ok(&["--master", master.to_str().unwrap(), "master", "list", "--
json"]);
    let v: serde_json::Value = serde_json::from_str(&stdout(&out)).unwrap();
    assert_eq!(v[0]["content_mode"], "search-only");
}
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cargo test --test cli master_list_json_carries_content_mode` Expected: FAIL — `v[0]["content_mode"]` is `Value::Null`, not `"search-only"`.

- [ ] **Step 3: Add the field to `master list --json`**

In `src/main.rs`, `MasterCommands::List { json }` (around line 288-300):

```
            if json {
                let v: Vec<serde_json::Value> = rows
                    .iter()
                    .map(|r| {
                        serde_json::json!({
                            "archive_id": r.archive_id, "label": r.label,
                            "source": r.source_path, "source_type": r.source_type,
                            "db_path": r.db_path, "schema_version":
r.schema_version,
                            "files": r.files_count, "completed": r.completed,
                            "status": r.status, "indexed_unix": r.indexed_unix,
                            "content_mode": r.content_mode,
                        })
                    })
                    .collect();
```

- [ ] **Step 4: Add the `Mode` column to the text table**

In `src/main.rs`, `print_master_table` (around line 699-730):

```
fn print_master_table(rows: &[master::ArchiveRow]) {
    let mut table = Table::new();
    table.set_content_arrangement(ContentArrangement::Dynamic);
    table.set_header(vec![
        header_cell("ID"),
        header_cell("Label"),
        header_cell("Type"),
        header_cell("Files"),
        header_cell("Schema"),
        header_cell("Mode"),
        header_cell("Status"),
        header_cell("Indexed"),
        header_cell("Source"),
    ]);
    for r in rows {
        let status_color = match r.status.as_str() {
            "ok" => Color::Green,
            "v2-limited" | "incomplete" => Color::Yellow,
            _ => Color::Red,
        };
        table.add_row(vec![
            Cell::new(r.archive_id).set_alignment(CellAlignment::Right),
            Cell::new(sanitize(&r.label)).add_attribute(Attribute::Bold),
            Cell::new(sanitize(&r.source_type)),

Cell::new(format_number(r.files_count)).set_alignment(CellAlignment::Right),
            Cell::new(format!("v{}", r.schema_version)),
            Cell::new(sanitize(&r.content_mode)),
            Cell::new(sanitize(&r.status)).fg(status_color),
            Cell::new(r.indexed_unix.map(fmt_unix).unwrap_or_else(|| "-".into())),
            Cell::new(sanitize(&r.source_path)).fg(Color::DarkGrey),
        ]);
    }
    println!("{table}");
}
```

Note: `print_master_table` is also called from `MasterCommands::Verify`'s text branch (`ArchiveRow` returned by `Master::verify`, which reuses the same `list()` shape) — no other call site needs changes since the function signature is unchanged.

- [ ] **Step 5: Run test to verify it passes**

Run: `cargo test --test cli master_list_json_carries_content_mode` Expected: PASS

- [ ] **Step 6: Commit**

```
git add src/main.rs tests/cli.rs
git commit -m "feat(#71): show content_mode on master list (table + json)"
```

Task 3: Close the dedup silent-empty gap for metadata-only archives

Files: - Modify: `src/report.rs:32-39` (`ReportArchive`), `src/dedup.rs:404-427` (`Summary` assembly), `src/dedup.rs:448-458` (`archives:` assembly) - Test: `tests/dedup.rs` (new test appended)

Interfaces: - Consumes: `ArchiveRow.content_mode` (Task 1). - Produces: `ReportArchive.content_mode: String` (additive on `dedup --json`); `Summary.skipped_archives` now also contains `(label, reason)` pairs for metadata-only archives, so `has_skips()` (and therefore exit code 2) fires for a metadata-only-only master where it previously silently reported zero groups with exit 0.

- [ ] **Step 1: Write the failing test**

Append to `tests/dedup.rs`:

```
/// #71: before this fix, a metadata-only archive (no content_hash on any
/// row) simply vanished from `fetch_scope`'s `WHERE content_hash IS NOT
/// NULL` filter — dedup reported zero groups, exit 0, no explanation. It
/// must now land in `summary.skipped_archives` with a worded reason and
/// force exit 2, the same shape v2-limited archives already use.
#[test]
fn metadata_only_archive_is_a_counted_skip_not_a_silent_empty_report() {
    let dir = tempfile::tempdir().unwrap();
    let archive = write_archive(
        dir.path(),
        "mo.tar",
        &build_tar(&[("secret.txt", b"never hashed".to_vec())]),
    );
    let opts = backupsage::indexer::IndexOptions {
        mode: backupsage::indexer::ContentMode::MetadataOnly,
        ..backupsage::indexer::IndexOptions::default()
    };
    let db = backupsage::indexer::run_index(&archive, None, &opts)
        .unwrap()
        .db_path;
    let mut m = backupsage::master::open_at(&dir.path().join("m.db")).unwrap();
    m.add(&db).unwrap();

    let report = backupsage::dedup::run_dedup(&m,
&backupsage::dedup::DedupParams::default())
        .unwrap();
    assert!(report.has_skips(), "metadata-only archive must count as a skip");
    assert_eq!(report.summary.skipped_archives.len(), 1);
    assert_eq!(report.summary.skipped_archives[0].0, "mo.tar");
    assert!(
        report.summary.skipped_archives[0].1.contains("metadata-only"),
        "{}",
        report.summary.skipped_archives[0].1
    );
    assert_eq!(report.archives[0].content_mode, "metadata-only");
}
```

- [ ] **Step 2: Run test to verify it fails**

Run: `cargo test --test dedup metadata_only_archive_is_a_counted_skip` Expected: FAIL — `report.has_skips()` is `false` (empty `skipped_archives`), and `report.archives[0].content_mode` is a compile error (`ReportArchive` has no such field yet) — fix the compile error first by adding the field (next step), then the test will compile and show the assertion failure.

- [ ] **Step 3: Add the field to `ReportArchive`**

In `src/report.rs` (around line 32-39):

```
#[derive(Debug, Serialize)]
pub struct ReportArchive {
    pub archive_id: i64,
    pub label: String,
    pub source: String,
    pub source_type: String,
    pub status: String,
    pub content_mode: String,
}
```

- [ ] **Step 4: Populate it and extend the skip logic in `dedup.rs`**

In `src/dedup.rs`, the `Summary` assembly (around line 398-430) — extend `skipped_archives` to also cover metadata-only archives, keeping the existing v2-limited entries:

```
    let in_scope = |id: i64| scope_ids.contains(&id);
    let summary = Summary {
        groups: groups.len(),
        duplicate_files,
        reclaimable_bytes: reclaimable_total,
        transitive_only_files,
        review_only_bytes: review_only_total,
        archives_offline: registry
            .iter()
            .filter(|a| in_scope(a.archive_id) && a.status == STATUS_DB_MISSING)
            .map(|a| a.label.clone())
            .collect(),
        archives_incomplete: registry
            .iter()
            .filter(|a| in_scope(a.archive_id) && a.status == STATUS_INCOMPLETE)
            .map(|a| a.label.clone())
            .collect(),
        skipped_archives: registry
            .iter()
            .filter(|a| in_scope(a.archive_id))
            .filter_map(|a| {
                if a.status == STATUS_V2_LIMITED {
                    Some((
                        a.label.clone(),
                        format!(
                            "v2-limited — no hashes; run `backupsage index {}` to
upgrade",
                            a.source_path
                        ),
                    ))
                } else if a.content_mode ==
crate::indexer::ContentMode::MetadataOnly.as_str() {
                    Some((
                        a.label.clone(),
                        "metadata-only — no hashes; re-index with --mode full or
search-only"
                            .into(),
                    ))
                } else {
                    None
                }
            })
            .collect(),
        images_without_phash,
        intra_archive_shadowed_bytes: shadowed_bytes,
        near_buckets_skipped,
    };
```

And extend the `archives:` field in the returned `DedupReport` (around line 448-458):

```
        archives: registry
            .iter()
            .filter(|a| in_scope(a.archive_id))
            .map(|a| ReportArchive {
                archive_id: a.archive_id,
                label: a.label.clone(),
                source: a.source_path.clone(),
                source_type: a.source_type.clone(),
                status: a.status.clone(),
                content_mode: a.content_mode.clone(),
            })
            .collect(),
```

- [ ] **Step 5: Run test to verify it passes**

Run: `cargo test --test dedup` Expected: PASS (all existing `tests/dedup.rs` tests plus the new one — check specifically that no existing test asserted `skipped_archives` is empty on a corpus that happens to include a metadata-only archive; grep confirms none do today).

- [ ] **Step 6: Commit**

```
git add src/report.rs src/dedup.rs tests/dedup.rs
git commit -m "fix(#71): metadata-only archives are a counted dedup skip, not a
silent empty report"
```

Task 4: `content_mode` on `search --all` (per-archive `mode` field + snippets note)

Files: - Modify: `src/searcher.rs:236-300` (`FederatedHit`, `search_all`), `src/main.rs:69-96` (`Commands::Search(args) if args.all`), `src/main.rs:744-765` (`search_all_json`) - Test: `tests/cli.rs` (new tests appended)

Interfaces: - Consumes: `crate::store::content_mode(&Connection)` (existing). - Produces: `FederatedHit.content_mode: String`; `search --all --json` per-archive objects gain `"mode"` (same key name as the existing top-level `mode` field on single-archive `search --json`, #70).

- [ ] **Step 1: Write the failing tests**

Append to `tests/cli.rs`:

```
#[test]
fn search_all_json_carries_per_archive_mode() {
    let dir = tempfile::tempdir().unwrap();
    let master = dir.path().join("m.db");
    let a = write_archive(
        dir.path(),
        "full.tar",
        &build_tar(&[("a.txt", b"findableword here".to_vec())]),
    );
    let b = write_archive(
        dir.path(),
        "so.tar",
        &build_tar(&[("b.txt", b"findableword too".to_vec())]),
    );
    run_ok(&["index", a.to_str().unwrap()]);
    run_ok(&["index", b.to_str().unwrap(), "--mode", "search-only"]);
    run_ok(&[
        "--master",
        master.to_str().unwrap(),
        "master",
        "add",
        dir.path().join("full.tar.db").to_str().unwrap(),
        dir.path().join("so.tar.db").to_str().unwrap(),
    ]);
    let out = run_ok(&[
        "--master",
        master.to_str().unwrap(),
        "search",
        "findableword",
        "--all",
        "--json",
    ]);
    let v: serde_json::Value = serde_json::from_str(&stdout(&out)).unwrap();
    let archives = v["archives"].as_array().unwrap();
    let modes: std::collections::BTreeSet<&str> = archives
        .iter()
        .map(|a| a["mode"].as_str().unwrap())
        .collect();
    assert_eq!(
        modes,
        std::collections::BTreeSet::from(["full", "search-only"])
    );
}

#[test]
fn search_all_snippets_notes_search_only_archives() {
    let dir = tempfile::tempdir().unwrap();
    let master = dir.path().join("m.db");
    let b = write_archive(
        dir.path(),
        "so2.tar",
        &build_tar(&[("b.txt", b"snippetword content".to_vec())]),
```

```
    );
    run_ok(&["index", b.to_str().unwrap(), "--mode", "search-only"]);
    run_ok(&[
        "--master",
        master.to_str().unwrap(),
        "master",
        "add",
        dir.path().join("so2.tar.db").to_str().unwrap(),
    ]);
    let out = bin()
        .args([
            "--master",
            master.to_str().unwrap(),
            "search",
            "snippetword",
            "--all",
            "--snippets",
        ])
        .output()
        .unwrap();
    let err = String::from_utf8_lossy(&out.stderr);
    assert!(err.contains("search-only"), "{err}");
    assert!(err.contains("so2.tar"), "{err}");
}
```

- [ ] **Step 2: Run tests to verify they fail**

Run: `cargo test --test cli search_all_json_carries_per_archive_mode search_all_snippets_notes_search_only_archives` Expected: first FAILs (`a["mode"]` is `Value::Null`), second FAILs (empty stderr — no note printed today).

- [ ] **Step 3: Add `content_mode` to `FederatedHit` and populate it in `search_all`**

In `src/searcher.rs` (around line 236-240):

```
pub struct FederatedHit {
    pub archive_label: String,
    pub hits: Vec<SearchHit>,
    pub truncated: bool,
    pub content_mode: String,
}
```

And in `search_all` (around line 288-298), capture the mode before running the search (it's already computed once for the metadata-only gate just above — reuse it rather than querying twice):

```
        // Metadata-only archives have no content to match — skip with a
        // worded reason (exit-2 semantics) instead of erroring out (#39).
        let mode = crate::store::content_mode(&conn);
        if mode == crate::indexer::ContentMode::MetadataOnly {
            out.skipped.push((
                row.label.clone(),
                "metadata-only — content search unsupported; re-index with \
                 --mode full or search-only"
                    .into(),
            ));
            continue;
        }
        let outcome = search(&conn, query, limit_per_archive, snippets)
            .with_context(|| format!("search failed in '{}'", row.label))?;
        if !outcome.hits.is_empty() {
            out.per_archive.push(FederatedHit {
                archive_label: row.label.clone(),
                hits: outcome.hits,
                truncated: outcome.truncated,
                content_mode: mode.as_str().to_string(),
            });
        }
```

- [ ] **Step 4: Add the `mode` field to `search_all_json`**

In `src/main.rs`, `search_all_json` (around line 744-765):

```
fn search_all_json(outcome: &searcher::FederatedOutcome) -> String {
    let archives: Vec<serde_json::Value> = outcome
        .per_archive
        .iter()
        .map(|f| {
            serde_json::json!({
                "archive": f.archive_label,
                "truncated": f.truncated,
                "mode": f.content_mode,
                "hits": f.hits.iter().map(hit_json).collect::<Vec<_>>(),
            })
        })
        .collect();
    let skipped: Vec<serde_json::Value> = outcome
        .skipped
        .iter()
        .map(|(l, r)| serde_json::json!({"archive": l, "reason": r}))
        .collect();
    serde_json::to_string_pretty(&serde_json::json!({
        "archives": archives, "skipped": skipped,
    }))
    .expect("json render cannot fail")
}
```

- [ ] **Step 5: Print the search-only snippets note in the `--all` branch**

In `src/main.rs`, `Commands::Search(args) if args.all` (around line 69-96), print the note for every search-only archive when `--snippets` was requested — mirroring the single-archive note at line 104-107, and placed before the json/text branch so it fires for both output modes exactly like that precedent:

```
        Commands::Search(args) if args.all => {
            let m = open_master_checked(&master_path)?;
            let outcome = searcher::search_all(&m, &args.keyword, args.limit,
args.snippets)?;
            if args.snippets {
                for fed in &outcome.per_archive {
                    if fed.content_mode == "search-only" {
                        eprintln!(
                            "note: '{}' is search-only — snippets are unavailable
(no stored text)",
                            sanitize(&fed.archive_label)
                        );
                    }
                }
            }
            if args.json {
                println!("{}", search_all_json(&outcome));
            } else {
```

(The rest of the `if args.json { ... } else { ... }` block is unchanged — only the new `if args.snippets { ... }` block is inserted above it.)

- [ ] **Step 6: Run tests to verify they pass**

Run: `cargo test --test cli search_all_json_carries_per_archive_mode search_all_snippets_notes_search_only_archives` Expected: PASS

- [ ] **Step 7: Commit**

```
git add src/searcher.rs src/main.rs tests/cli.rs
git commit -m "feat(#71): per-archive mode field and search-only snippets note on
search --all"
```

Task 5: Re-bless the five existing golden fixtures (additive-only)

Files: - Modify (generated, not hand-edited): `tests/fixtures/contract/master_list.json`, `tests/fixtures/contract/search_all.json`, `tests/fixtures/contract/dedup.json`, `tests/fixtures/contract/dedup_chain.json`, `tests/fixtures/contract/dedup_skips.json` - No modify needed: `tests/fixtures/contract/search.json` (already carries `mode` from #70), `tests/fixtures/contract/search_all_skips.json` (only `skipped[]` populated in that fixture — no `archives[]` entries to gain a `mode` field), `tests/fixtures/contract/master_verify.json` (verify's JSON shape is untouched by this issue)

Interfaces: - Consumes: Tasks 1-4 (the fields now exist to be frozen). - Produces: committed fixture diffs that must be additive-only — reviewed by hand in Step 3 before committing.

- [ ] **Step 1: Run the contract test to see it fail on drift**

Run: `cargo test --test contract json_surfaces_match_golden_fixtures dedup_chain_json_matches_fixture` Expected: FAIL — `golden fixture 'master_list.json' drifted` (new `content_mode` key), same for `search_all.json` (new `mode` key per archive) and the three dedup fixtures (new `content_mode` key per `archives[]` entry).

- [ ] **Step 2: Re-bless**

Run: `BACKUPSAGE_BLESS=1 cargo test --test contract json_surfaces_match_golden_fixtures dedup_chain_json_matches_fixture`

- [ ] **Step 3: Review the diff is additive-only**

Run: `git diff tests/fixtures/contract/` Expected: every hunk is a new line adding `"content_mode": "..."` or `"mode": "..."` — no existing key renamed, removed, or reordered in a way that changes its value. If anything else changed, stop and investigate before proceeding (a non-additive diff here is exactly the failure mode `docs/CONTRACT.md` exists to catch).

- [ ] **Step 4: Run the full contract suite to confirm green**

Run: `cargo test --test contract` Expected: PASS

- [ ] **Step 5: Commit**

```
git add tests/fixtures/contract/master_list.json tests/fixtures/contract/
search_all.json \
        tests/fixtures/contract/dedup.json tests/fixtures/contract/dedup_chain.json
\
        tests/fixtures/contract/dedup_skips.json
git commit -m "test(#71): re-bless golden fixtures with additive content_mode/mode
fields"
```

Task 6: New fixtures pinning non-degenerate `search-only` and `metadata-only` shapes

Files: - Modify: `tests/contract.rs` (two new `#[test]` functions, and the orphan-census array) - Create (generated, not hand-edited): `tests/fixtures/contract/search_only.json`, `tests/fixtures/contract/metadata_only_skips.json`

Interfaces: - Consumes: Tasks 1-4. - Produces: two new frozen fixtures joining the contract set (now ten total), each pinning a mode field at a real non-`"full"` value — per ADR 0003's residual-gaps note that a field frozen only at its degenerate/default value does not pin its real shape (the same reasoning `dedup_chain.json` already applies to `actionable`/`hamming_to_keep`).

- [ ] **Step 1: Write the two new fixture-generating tests**

Append to `tests/contract.rs`:

```
/// #71: `search --all --json`'s per-archive `mode` field pinned at a real
/// non-default value (`search-only`), not just `full` — ADR 0003's
/// residual-gaps rule: a field frozen only at its default is not really
/// pinned.
#[test]
fn search_all_search_only_mode_matches_fixture() {
    let dir = tempfile::tempdir().unwrap();
    let tmp = dir.path();
    let archive = write_archive(
        tmp,
        "so-contract.tar",
        &build_tar(&[("doc.txt", b"contractword body text".to_vec())]),
    );
    let out = run(&["index", archive.to_str().unwrap(), "--mode", "search-only"]);
    assert_eq!(code(&out), 0, "index failed: {}", stdout(&out));
    let master = tmp.join("so-master.db");
    let m = master.to_str().unwrap();
    let out = run(&[
        "--master",
        m,
        "master",
        "add",
        archive.with_extension("tar.db").to_str().unwrap(),
    ]);
    assert_eq!(code(&out), 0, "master add failed");

    let out = run(&["--master", m, "search", "contractword", "--all", "--json"]);
    assert_eq!(code(&out), 0);
    let report = normalized(&stdout(&out), tmp);
    assert_eq!(
        report["archives"][0]["mode"], "search-only",
        "fixture must pin a non-degenerate mode value: {report}"
    );
    assert_matches_fixture("search_only.json", report);
}

/// #71: `dedup --json` with a metadata-only archive present alongside a
/// normal duplicate pair — pins `summary.skipped_archives`' worded reason
/// and `archives[].content_mode` at real non-`full` values, and freezes
/// the exit-2 behavior of the previously-silent-empty gap.
#[test]
fn dedup_metadata_only_skip_matches_fixture() {
    let dir = tempfile::tempdir().unwrap();
    let tmp = dir.path();
    let dup = b"metadata-fixture duplicate payload\n".to_vec();
    let full = write_archive(
        tmp,
        "mo-full.tar",
        &build_tar(&[
            ("keep/one.bin", dup.clone()),
            ("keep/two.bin", dup),
        ]),
```

```
    );
    let metadata_only = write_archive(
        tmp,
        "mo-only.tar",
        &build_tar(&[("secret/three.bin", b"never hashed content".to_vec())]),
    );
    let out = run(&["index", full.to_str().unwrap()]);
    assert_eq!(code(&out), 0, "index failed: {}", stdout(&out));
    let out = run(&[
        "index",
        metadata_only.to_str().unwrap(),
        "--mode",
        "metadata-only",
    ]);
    assert_eq!(code(&out), 0, "index failed: {}", stdout(&out));
    let master = tmp.join("mo-master.db");
    let m = master.to_str().unwrap();
    let out = run(&[
        "--master",
        m,
        "master",
        "add",
        full.with_extension("tar.db").to_str().unwrap(),
        metadata_only.with_extension("tar.db").to_str().unwrap(),
    ]);
    assert_eq!(code(&out), 0, "master add failed");

    let out = run(&["--master", m, "dedup", "--json"]);
    assert_eq!(code(&out), 2, "metadata-only archive must force exit 2");
    let report = normalized(&stdout(&out), tmp);
    assert_eq!(report["summary"]["groups"], 1, "the full-mode pair still dedups:
{report}");
    assert_eq!(
        report["summary"]["skipped_archives"][0][0], "mo-only.tar",
        "{report}"
    );
    assert!(
        report["summary"]["skipped_archives"][0][1]
            .as_str()
            .unwrap()
            .contains("metadata-only"),
        "{report}"
    );
    assert_matches_fixture("metadata_only_skips.json", report);
}
```

- [ ] **Step 2: Run tests to verify they fail on the missing fixture**

Run: `cargo test --test contract search_all_search_only_mode_matches_fixture dedup_metadata_only_skip_matches_fixture` Expected: FAIL — `missing golden fixture ...`

`generate it with BACKUPSAGE_BLESS=1 cargo test --test contract` (and the orphan-census assertion later in this same task also fails until Step 4).

- [ ] **Step 3: Generate the fixtures**

Run: `BACKUPSAGE_BLESS=1 cargo test --test contract search_all_search_only_mode_matches_fixture dedup_metadata_only_skip_matches_fixture`

- [ ] **Step 4: Update the orphan-census fixture-set assertion**

In `tests/contract.rs`, `json_surfaces_match_golden_fixtures`'s trailing block (around line 280-300), add the two new names in alphabetical order:

```
        assert_eq!(
            names,
            [
                "dedup.json",
                "dedup_chain.json",
                "dedup_skips.json",
                "master_list.json",
                "master_verify.json",
                "metadata_only_skips.json",
                "search.json",
                "search_all.json",
                "search_all_skips.json",
                "search_only.json",
            ],
            "unexpected fixture set — remove orphans or update this list"
        );
```

Also update the doc comment at the top of `tests/contract.rs` (lines 4-8) from "eight fixtures total" to "ten fixtures total", and add one sentence describing the two new ones, matching the existing style:

```
//! The five JSON surfaces: `dedup --json`, `search --json`,
//! `search --all --json`, `master list --json`, `master verify --json` —
//! plus completed-with-skips variants of the two that have one, plus the
//! keeper-star chain corpus (#9) pinned in `dedup_chain.json`, plus #71's
//! non-degenerate content-mode corpora (`search_only.json`,
//! `metadata_only_skips.json`) — ten fixtures total.
```

- [ ] **Step 5: Run the full contract suite to confirm green**

Run: `cargo test --test contract` Expected: PASS (all ten fixtures present and matching, exit-code matrix unaffected).

- [ ] **Step 6: Commit**

```
git add tests/contract.rs tests/fixtures/contract/search_only.json \
        tests/fixtures/contract/metadata_only_skips.json
git commit -m "test(#71): freeze search-only and metadata-only content-mode
fixtures"
```

Task 7: ADR 0005, CONTRACT.md, COMPATIBILITY.md, README security note

Files: - Create: `docs/adr/0005-content-mode-registry.md` - Modify: `docs/adr/README.md` (index), `docs/CONTRACT.md`, `docs/COMPATIBILITY.md`, `README.md:176-182`

Interfaces: None (documentation only) — Consumes the shipped behavior from Tasks 1-6 to describe accurately.

- [ ] **Step 1: Check the ADR index format**

Run: `cat docs/adr/README.md` — confirm the existing one-line-per-ADR index format before appending (ADR 0001-0004 precedent).

- [ ] **Step 2: Write ADR 0005**

Create `docs/adr/0005-content-mode-registry.md`:

ADR 0005 — content_mode as a replicated fact, not a per-surface re-derivation

Date: 2026-08-13 · Status: accepted · Milestone: v1.0.2 (issue #71, child of #39)

Context

#70 introduced `ContentMode` (`full`/`search-only`/`metadata-only`) and stored it as a `content_mode` meta key on each per-source index, with three read-time gates (`search`, `top`, snippet availability) keyed off `store::content_mode`. ADR 0002's consequences section anticipated exactly this follow-up: "Future schema metadata cleanup (v1.0.2 'explicit index modes') should fold the capability marker into a proper schema registry." #70 explicitly scoped master/dedup replication out ("Master/dedup replication is #71", `tests/modes.rs:2`) because the master catalog has no notion of content mode at all — `dedup`'s `fetch_scope` filters on `content_hash IS NOT NULL`, so a metadata-only archive (which never hashes content by design) simply produced zero rows with no explanation, and `master list`/`search --all --json` had no way to say which mode an archive was built in without opening its per-source `.db` directly.

Decision

- **Replicate the fact once, at `Master::add()` time**, into a new `archives.content_mode` column — same probe-then-ALTER migration shape as `path_raw` (ADR 0002): new masters get the column in `CREATE TABLE` directly, masters opened from an on-disk file predating this change get it via `ALTER TABLE ... ADD COLUMN content_mode TEXT NOT NULL DEFAULT 'full'`. It is derived via the existing `store::content_mode` grandfather logic (absent meta key ⇒ `full`), not re-implemented.
- **Every consuming surface gets one additive field, named to match its neighbors.** `master list` (table + `--json`) and `dedup --json`'s `ReportArchive` use `content_mode` (matching `ArchiveRow`'s own field name and the existing `schema_version`/`status` style). `search --json` and `search --all --json` use `mode` (matching the top-level `mode` field #70 already put on single-archive `search --json`, so the two search surfaces read consistently).
- **Metadata-only archives become a counted dedup skip**, not a silent empty report. `Summary.skipped_archives` — until now populated only by `v2-limited` archives — gains a second, additive case: any archive whose `content_mode` is `metadata-only`. This closes the exact gap named in the issue: `fetch_scope`'s `content_hash IS NOT NULL` filter (`src/dedup.rs`) already excluded these rows correctly; what was missing was telling the caller why the count is zero. `has_skips()` (and therefore exit code 2) now fires for a metadata-only-only master, matching the existing v2-limited exit-code contract.
- **`search --all` gets the same search-only snippets note the single-archive path already had.** `search()` (single-index) has printed `"note: search-only index — snippets are unavailable"` since #70; the federated path silently dropped snippets with no note. `FederatedHit` now carries `content_mode`, and the CLI prints the same note per search-only archive when `--snippets` was requested.

```
## Alternatives considered

- **Re-derive `content_mode` per surface by opening each source `.db` at
  render time** — rejected: the master's whole design point is answering
  registry-shaped questions (`list`, `dedup`, federated `search`) without
  every source database being reachable (`archives_offline` exists
  precisely because sources go offline); re-deriving would silently lose
  the mode fact for exactly the archives most likely to need it explained.
- **One shared field name (`mode`) everywhere** — rejected: `master list`
  and `dedup`'s `ReportArchive` already name their other replicated facts
  after the `ArchiveRow` column (`schema_version`, `status`, `source_type`);
  breaking that pattern for one field to match `search --json`'s unrelated
  precedent would make the two surfaces internally inconsistent instead of
  externally consistent. Naming to match each surface's *existing* sibling
  fields costs nothing and reads better in both places.
- **A full mode *registry* table (id → capabilities bitmap), per ADR 0002's
  literal wording** — rejected as premature: there are exactly three modes,
  they do not vary independently (each already has one canonical
  capability set enforced in `store.rs`/`searcher.rs`), and a registry
  table buys nothing until a fourth mode or per-mode-capability query
  actually exists. The `content_mode: TEXT` column is that registry's
  degenerate/correct-for-now form; revisit only if that changes.

## Consequences

- `master list`, `dedup --json`, and `search --all --json` all gained one
  additive field each; `docs/CONTRACT.md` and `docs/COMPATIBILITY.md`
  record the exact keys. No existing field renamed, removed, or changed
  type — the fixture re-bless diffs are additions only.
- A metadata-only-only master now exits 2 from `dedup` where it previously
  exited 0 with an empty report. This is a **behavior change** within the
  documented exit-code contract's own terms (`2` = "completed with skips"),
  not a new exit code or a new meaning for an existing one — scripts already
  treating exit 2 as "check `skipped_archives`" see one more legitimate
  reason to.
- `tests/fixtures/contract/search_only.json` and `metadata_only_skips.json`
  join the frozen set (ten fixtures total) so both non-`full` modes are
  pinned at real values on at least one surface each, per ADR 0003's
  residual-gaps note.
- The two-line README security note (search-only reduces stored content but
  is not encryption; metadata-only stores no content-derived data) is now
  the single place that states the privacy posture of all three modes
  together, closing the gap where #70 documented the read-time behavior but
  not the honest security framing.
```

- [ ] **Step 3: Add ADR 0005 to the index**

Append one line to `docs/adr/README.md` matching the existing format for ADR 0001-0004.

- [ ] **Step 4: Update `docs/CONTRACT.md`**

Three edits to the JSON surfaces table (around line 33-38):

Replace the `dedup --json` row's description to add, after the existing v1.0.2 (#9) sentence: `Also added \`ReportArchive.content_mode` and a metadata-only case in `summary.skipped_archives` (#71) — additive, still version 1.`

Replace the `search --all --json` row's last sentence (`The per-archive \`mode` field is #71.`) with:` The per-archive `mode` field (#71) is now present on every successfully-searched archive.`

Replace the `master list --json` row's shape column to add `content_mode` to the field list, and its description to: `**Frozen-as-observed.** v1.0.2 (#71) added \`content_mode` — additive.`

And update the fixture count in the "Golden fixtures" section intro (around line 11-15) from "eight fixtures" to "ten fixtures", listing the two new ones alongside the existing description, matching Task 6 Step 4's wording in `tests/contract.rs`.

- [ ] **Step 5: Update `docs/COMPATIBILITY.md`**

Append one sentence to the "JSON reports" row (around line 21), after the existing #70 sentence: `v1.0.2 also added \`content_mode` to `master list --json` and `dedup --json`'s `archives[]`, and `mode` to `search --all --json`'s `archives[]` (#71) — additive; a metadata-only archive now also appears in `dedup`'s `summary.skipped_archives` (previously silently produced zero groups with no explanation).`

- [ ] **Step 6: Update the README security note**

In `README.md` (around line 176-182), replace the "Honest numbers & limits" bullet that currently reads:

```
- **Index size**: the FTS index stores a full copy of all indexed *text*
  (that's what makes snippets work) — expect roughly the size of the text
  content. Media contribute only ~150 bytes of metadata per file. The
  security note stands: the `.db` contains plaintext from your backup —
  protect it like the backup itself.
```

with:

```
- **Index size**: the FTS index stores a full copy of all indexed *text*
  (that's what makes snippets work) — expect roughly the size of the text
  content. Media contribute only ~150 bytes of metadata per file. The
  security note stands: a `--mode full` (default) `.db` contains plaintext
  from your backup — protect it like the backup itself.
- **Content modes and what they leak** (`--mode full|search-only|metadata-only`,
  #39/#70/#71): `search-only` drops the recoverable-plaintext copy (SQLite
  FTS5 `contentless` storage) but still stores every distinct token and its
  frequency — that is what makes it searchable at all. **This is not
  encryption and not a privacy boundary**: word lists and frequencies can
  leak real information about the source text. `metadata-only` stores no
  content-derived data whatsoever — no hashes, no tokens, no snippets, no
  dedup — only filesystem-level facts (path, size, mtime, kind). Treat
  `search-only` indexes with the same care as `full` ones; only
  `metadata-only` indexes are safe to handle more casually than the backup
  they describe.
```

- [ ] **Step 7: Verify every doc reference resolves**

Run: `grep -rn "is #71\|(#71)" docs/ README.md` and confirm every remaining hit is a completed-tense description (e.g. "added ... (#71)"), not a forward-reference placeholder like "The per-archive `mode` field is #71."

- [ ] **Step 8: Commit**

```
git add docs/adr/0005-content-mode-registry.md docs/adr/README.md docs/CONTRACT.md
\
        docs/COMPATIBILITY.md README.md
git commit -m "docs(#71): ADR 0005, contract/compatibility updates, honest security
note"
```

Task 8: Full gate, PR, close #71 and #39

Files: None (verification and process only).

- [ ] **Step 1: Run the full gate**

Run: `cargo fmt --check && cargo clippy --all-targets -- -D warnings && cargo test`
Expected: PASS, zero warnings.

- [ ] **Step 2: Manual smoke test of the previously-silent gap**

```
dir=$(mktemp -d)
cd "$dir"
mkdir src && echo secret > src/a.txt
tar -cf a.tar src
backupsage index a.tar --mode metadata-only
backupsage --master m.db master add a.tar.db
backupsage --master m.db dedup --json | grep -A2 skipped_archives
echo "exit code: $?"
```

Expected: `skipped_archives` names `a.tar` with a `metadata-only` reason; the `dedup` invocation itself exits 2 (check with `backupsage --master m.db dedup >/dev/null; echo $?` separately, since the pipe above reports `grep`'s exit code).

- [ ] **Step 3: Push and open the PR**

```
git push -u origin issue-71-content-mode-surfaces
gh pr create --title "v1.0.2: replicate content_mode through master, dedup, search
(#71)" --body "$(cat <<'EOF'
## Summary
- Replicates `content_mode` (#70) into the master catalog
(`archives.content_mode`), probe-then-ALTER migrated like `path_raw` (ADR 0002).
- `master list` (table + `--json`), `dedup --json` (`ReportArchive.content_mode`),
and `search --all --json` (per-archive `mode`) all surface it — additive.
- Closes the dedup silent-empty gap: a metadata-only archive now lands in
`summary.skipped_archives` with a worded reason and forces exit 2, instead of
silently reporting zero groups.
- `search --all --snippets` now notes search-only archives, matching the single-
archive path's existing note.
- Two new golden fixtures (`search_only.json`, `metadata_only_skips.json`) pin both
non-`full` modes at non-degenerate values; five existing fixtures re-blessed
additive-only.
- ADR 0005, `docs/CONTRACT.md`, `docs/COMPATIBILITY.md`, and the README security
note document the leakage honesty for all three modes.

## Test plan
- [x] `cargo test` (full suite, including new unit/integration tests per task)
- [x] `cargo test --test contract` (ten golden fixtures, additive-diff reviewed by
hand)
- [x] `cargo clippy --all-targets -- -D warnings`, `cargo fmt --check`
- [x] Manual smoke test of the metadata-only dedup skip end-to-end

Closes #71.
EOF
)"
```

- [ ] **Step 4: After merge — close #39 and start #11**

Once this PR merges, #39 (parent, "Add explicit content indexing modes") has both children (#70, #71) closed — close it with a summary comment pointing at both PRs. Then move to #11 (v1 debt register closeout), the last open v1.0.2 item per the 2026-08-02 handoff.

Self-Review

Spec coverage (issue #71 acceptance criteria, checked against tasks above): - "Mode visible per archive in master list (text + JSON) and dedup report" → Task 2 (master list), Task 3 (`ReportArchive.content_mode`). - "Metadata-only never silently empties dedup/ search --all — always a counted, worded skip with exit 2" → Task 3 (dedup; search --all's metadata-only skip already existed from #70, confirmed in `src/searcher.rs:280-288` during research). - "All fixtures additive; migration fixture proves old-index

grandfathering" → Task 5 (re-bless), Task 1 Step 1 (`pre_70_index_without_content_mode_key_replicates_as_full`, the migration test — issue calls it a "fixture" but the established precedent, `v2_databases_register_limited_and_moves_reregister`, is a Rust integration test, not a JSON fixture file; matched that precedent). - "README/CONTRACT/COMPATIBILITY/ADR document leakage honestly" → Task 7. - Verifier polish items from the #70 handoff (federated `--snippets` note, README security sentence) → Task 4 Step 5, Task 7 Step 6.

Placeholder scan: every step has literal code, not a description of code; every test has real assertions; no "TBD"/"similar to Task N".

Type consistency: `ArchiveRow.content_mode: String` (Task 1) flows unchanged into `ReportArchive.content_mode: String` (Task 3, `a.content_mode.clone()`) and is read as `&str` for comparison in Task 3's skip filter (`a.content_mode == ContentMode::MetadataOnly.as_str()`). `FederatedHit.content_mode: String` (Task 4) is independently sourced from `store::content_mode(&conn).as_str().to_string()` — deliberately not threaded from `ArchiveRow`, since `search_all` opens each source `.db` directly and already had the connection open for the metadata-only gate check; reusing that connection's live read is more honest than trusting a possibly-stale master replica for a per-query decision. Both spellings agree at every call site checked in Task 3/4's steps.